

# SYNCHRONOUS FIFO VERIFICATION

SystemVerilog Verification Environment

## VERIFICATION DIARY & DEBUG LOGBOOK

**Project Name:** Synchronous FIFO Verification  
**Document Type:** Verification Debug Diary / Bug Tracker  
**Author:** Trần Hoàng An  
**Repository:** synchronous-fifo-verification

# LOG ENTRY #01: Scoreboard Sampling Read Output Issue

## ISSUE METADATA & TRACKING INFO

**Date Logged:** 22/07/2026  
**Issue ID:** BUG-20260722-01  
**Component:** Testbench / Mini Scoreboard (`queue_test.sv`)  
**Severity:** Major (Simulation Failure / False Mismatch)  
**Status:** Resolved (Workaround Applied)

## Bối cảnh

Môi trường verification sử dụng monitor để quan sát các giao dịch đọc và ghi của FIFO. Sau đó, monitor tạo đối tượng `fifo_transaction` và gửi transaction đến scoreboard thông qua mailbox. Khi có một giao dịch đọc hợp lệ, scoreboard lấy dữ liệu mong đợi từ reference queue rồi so sánh với giá trị `dout` của DUT.

## Hiện tượng

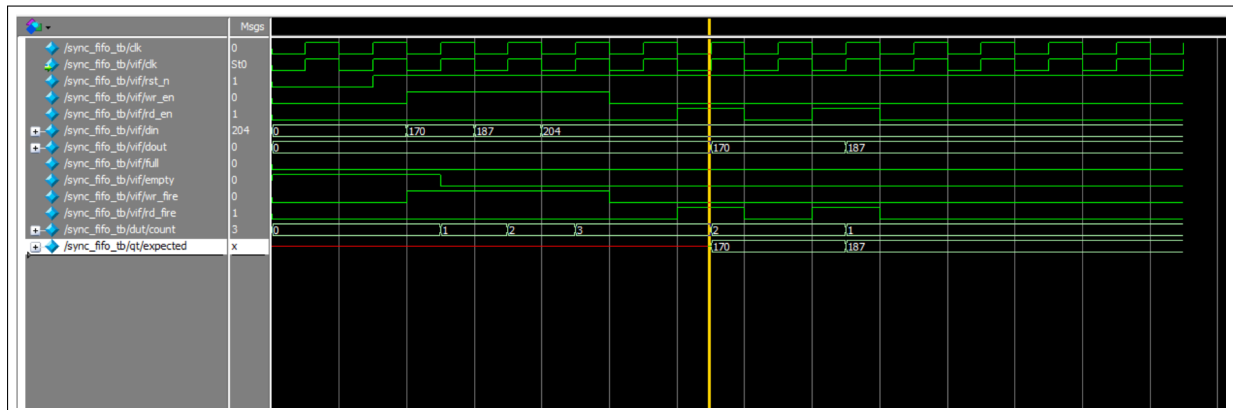
Khi kiểm tra FIFO bằng reference queue, scoreboard báo lỗi:

```
# -----  
# -----  
# WRITE: 170 | queue = '{170}'  
# -----  
# WRITE: 187 | queue = '{170, 187}'  
# -----  
# WRITE: 204 | queue = '{170, 187, 204}'  
# -----  
# -----  
# ** Error: FAIL: expected = 170 | vif.dout = 0  
#   Time: 65 ns  Scope: sync_fifo_tb.qt File: tb/queue_test.sv Line: 20  
# -----  
# -----  
# ** Error: FAIL: expected = 187 | vif.dout = 170  
#   Time: 85 ns  Scope: sync_fifo_tb.qt File: tb/queue_test.sv Line: 20  
# -----  
# -----  
# -----  
# -----  
# ** Note: $finish      : tb/sync_fifo_tb.sv(67)  
#   Time: 135 ns  Iteration: 1  Instance: /sync_fifo_tb
```

Hình 1: Ảnh chụp Console Log báo lỗi (Expected vs Actual lệch nhau).

Trong khi waveform cho thấy FIFO thực tế vẫn xuất dữ liệu đúng thứ tự:

170 -> 187 -> ...



Hình 2: Ảnh chụp Waveform cho thấy FIFO thực tế xuất dữ liệu đúng thứ tự.

Có thể nhận thấy rằng dout luôn chậm hơn **expected** một lần đọc (ví dụ Expected: BB, Actual: AA).

## Nguyên nhân

Checker được viết theo dạng:

```
always @(posedge vif.clk) begin
    if (vif.rd_fire) begin
        expected = ref_queue.pop_front();

        if (expected != vif.dout)
            $error(...);
    end
end
```

Trong DUT, dout được cập nhật bằng nonblocking assignment:

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        dout <= '0;
    else if (rd_en && !empty)
        dout <= mem[rd_ptr];
end
```

Tại cùng một cạnh lên của clock, monitor/checker đọc giá trị **dout** trước khi nonblocking assignment của DUT cập nhật giá trị mới ở vùng NBA region.

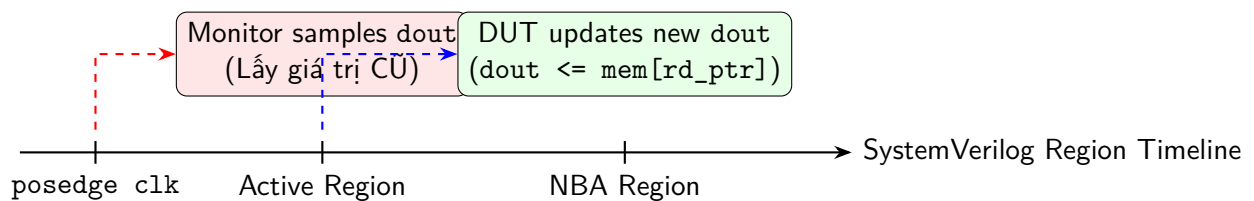
Trình tự xảy ra có thể hiểu như sau:

```
posedge clk
|
+-- Monitor/Checker đọc dout cũ (Active Region)
|
+-- DUT thực hiện: dout <= mem[rd_ptr]
|
+-- Cuối time step, dout mới được cập nhật (NBA Region)
```

Do đó:

```
READ 1:
expected = 170
dout      = 0      // giá trị cũ

READ 2:
expected = 187
dout      = 170    // kết quả của READ trước
```



Hình 3: Sơ đồ minh họa thứ tự lấy mẫu và cập nhật tín hiệu của SystemVerilog.

## Quá trình phân tích

Ban đầu, lỗi được nghi ngờ nằm trong scoreboard hoặc reference queue. Tuy nhiên, reference queue vẫn push và pop đúng thứ tự. Dữ liệu expected chính xác, nhưng trường dout bên trong transaction bị sai ngay từ lúc monitor tạo ra. Điều này khẳng định lỗi không nằm ở phép so sánh mà do thời điểm monitor lấy mẫu signal.

## Cách xử lý

Monitor không lấy mẫu dout ngay tại posedge clk. Sau khi phát hiện rd\_fire, monitor chờ đến negedge clk rồi mới đọc dout. Tại thời điểm này, giá trị gán bằng nonblocking assignment đã hoàn toàn ổn định:

```
always @(posedge vif.clk) begin
    if (vif.rd_fire) begin
        @(negedge vif.clk); // Chờ đến negedge để dout đã cập nhật xong ở NBA
        tr.dout = vif.dout;
        mon2scb.put(tr);
    end
end
```

## Kết quả sau khi sửa

Scoreboard nhận đúng dữ liệu của từng giao dịch đọc. Expected và actual không còn bị lệch một chu kỳ. các test case basic write/read và wrap-around đều vượt qua chính xác.

### Bài học rút ra

Đây không nhất thiết là lỗi của FIFO DUT mà là vấn đề về **thời điểm sampling dữ liệu trong testbench**.

Khi DUT sử dụng nonblocking assignment, giá trị output mới chỉ được cập nhật sau khi các khối kích hoạt tại cạnh clock hiện tại đã thực thi.

Vì vậy, trong verification cần chú ý:

- Thời điểm Driver drive tín hiệu.
- Thời điểm DUT sample input.
- Thời điểm DUT cập nhật output.
- Thời điểm Monitor hoặc Scoreboard sample output.

Clocking block và Monitor lấy mẫu tại **negedge** hoặc thông qua offset giúp kiểm soát thời điểm drive và sample tín hiệu, triệt tiêu hoàn toàn **sampling timing issue**.

## LOG ENTRY #02: Compilation Unit & Compile Order Dependency Error

### ISSUE METADATA & TRACKING INFO

**Date Logged:** 23/07/2026  
**Issue ID:** BUG-20260723-02  
**Component:** Build Environment / Compilation Scripts  
**Severity:** Critical (Compilation & Elaboration Failure)  
**Status:** Resolved

### Bối cảnh

Môi trường verification được chia thành nhiều file riêng biệt: `fifo_transaction.sv`, `fifo_monitor.sv`, `fifo_scoreboard.sv`, `fifo_if.sv`, `sync_fifo.sv`, và `fifo_tb.sv`. Monitor sử dụng class `fifo_transaction`. Testbench đồng thời instantiate interface `fifo_if` và DUT `sync_fifo`.

### Hiện tượng

Trong quá trình compile trên ModelSim xuất hiện nhiều lỗi không tìm thấy class hoặc design unit:

```
ModelSim> vlog -sv tb/fifo_monitor.sv
# Model Technology ModelSim - Intel FPGA Edition vlog 2020.1 Compiler 2020.02 Feb 28 2020
# Start time: 10:16:39 on Jul 26,2026
# vlog -reportprogress 300 -sv tb/fifo_monitor.sv
# ** Error: tb/fifo_monitor.sv(11): (vlog-2730) Undefined variable: 'fifo_transaction'.
# ** Error: (vlog-13069) tb/fifo_monitor.sv(11): near "(": syntax error, unexpected '(', expecting ')' or ','
# End time: 10:16:39 on Jul 26,2026, Elapsed time: 0:00:00
# Errors: 2, Warnings: 0
# D:/Linh_tinh/intelFPGA/20.1/modelsim_ase/win32aloem/vlog failed.
ModelSim>
```

Hình 4: Ảnh console ModelSim hiển thị lỗi `fifo_transaction` không được nhận diện.

```

ModelSim> vsim work.fifo_tb
# vsim work.fifo_tb
# Start time: 10:26:16 on Jul 26, 2026
# Loading sv_std.std
# Loading work.fifo_transaction_sv_unit
# Loading work.fifo_tb
# ** Error: (vsim-3033) Instantiation of 'fifo_if' failed. The design unit was not found.
#   Time: 0 ps  Iteration: 0  Instance: /fifo_tb File: tb/fifo_tb.sv Line: 10
#       Searched libraries:
#           D:/Project/Summer_Project/synchronous-fifo-verification/work
# ** Error: (vsim-3033) Instantiation of 'sync_fifo' failed. The design unit was not found.
#   Time: 0 ps  Iteration: 0  Instance: /fifo_tb File: tb/fifo_tb.sv Line: 15
#       Searched libraries:
#           D:/Project/Summer_Project/synchronous-fifo-verification/work
# Error loading design

```

Hình 5: Ảnh console ModelSim hiển thị lỗi không tìm thấy design unit `fifo_if` hoặc `sync_fifo`.

Các lỗi tiêu biểu bao gồm:

```

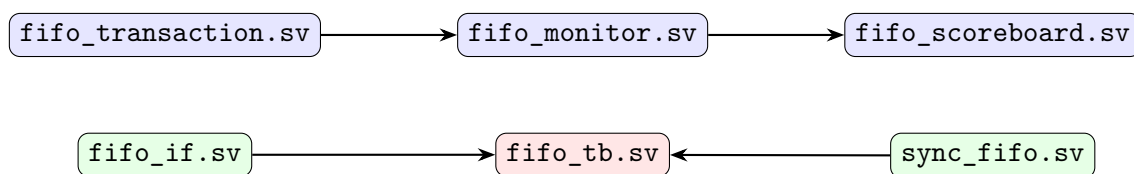
Undefined variable or unknown type: fifo_transaction
Instantiation of fifo_if failed. The design unit was not found.
Instantiation of sync_fifo failed. The design unit was not found.

```

## Nguyên nhân

Nguyên nhân chính nằm ở cách các file được compile và khả năng nhìn thấy nhau (visibility) giữa các compilation unit:

1. khi `fifo_transaction.sv` và `fifo_monitor.sv` biên dịch riêng lẻ, ModelSim xem chúng thuộc các compilation unit khác nhau. Vì class không nằm trong `package`, monitor không thể nhìn thấy định nghĩa class.
2. `fifo_if` và `sync_fifo` chưa được compile vào thư viện `work` trước khi `fifo_tb.sv` elaboration.
3. Tên file trong lệnh compile bị lệch với tên thật (ví dụ lệnh biên dịch tìm `tb/sync_fifo_scoreboard` nhưng file thực tế là `tb/fifo_scoreboard.sv`).



Hình 6: Sơ đồ phụ thuộc thứ tự biên dịch (File bên trái phải biên dịch trước file bên phải).

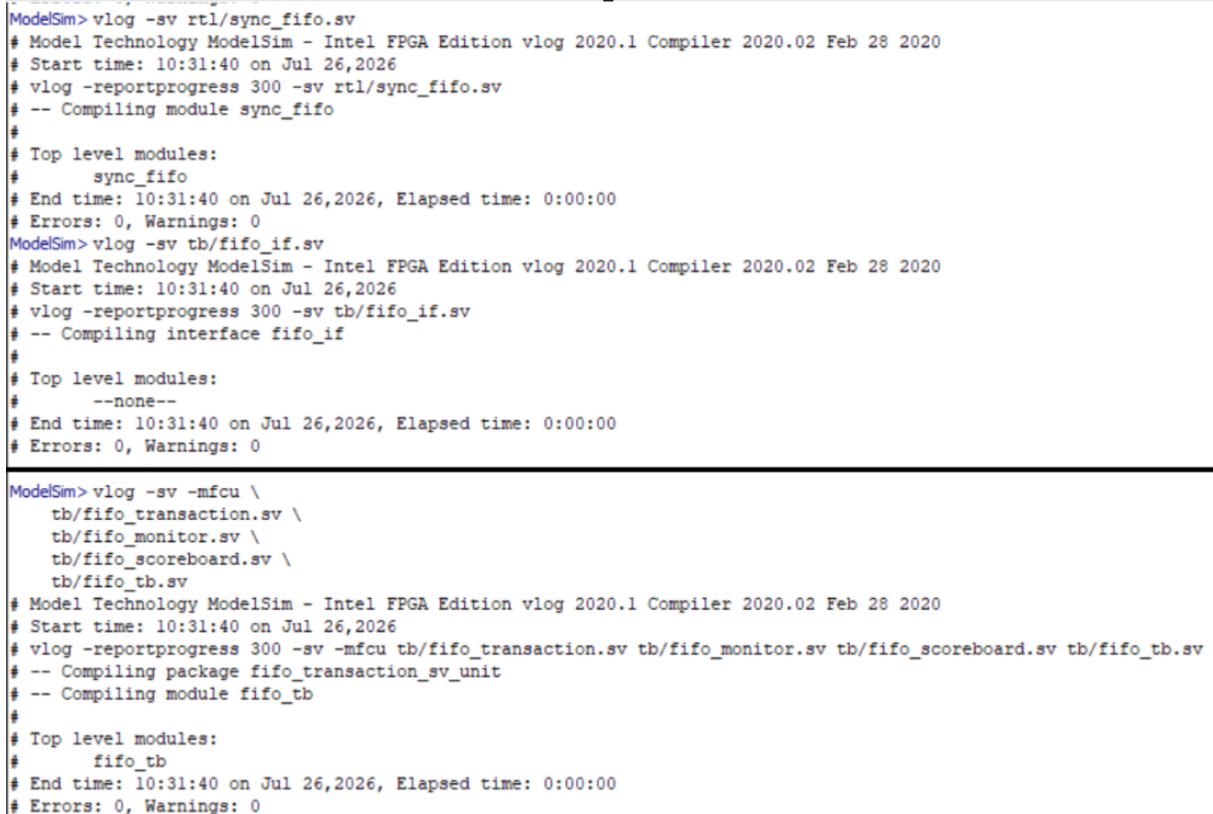
## Quá trình phân tích

Ban đầu, lỗi nghi ngờ do cú pháp typed mailbox không tương thích. Tuy nhiên, việc kiểm tra thư viện `work` cho thấy module và interface chưa tồn tại ở thời điểm elaboration. Lỗi hoàn toàn xuất phát từ script build và compilation unit dependency.

## Cách khắc phục

Sử dụng tùy chọn `-mfcu` (Multi-File Compilation Unit) để compile các file class chung một unit, chuẩn hóa thứ tự biên dịch RTL/Interface trước Testbench, và sửa đường dẫn file:

```
vlib work
vlog -sv rtl/sync_fifo.sv
vlog -sv tb/fifo_if.sv
vlog -sv -mfcu tb/fifo_transaction.sv tb/fifo_monitor.sv tb/fifo_scoreboard.sv
vlog -sv tb/fifo_tb.sv
vsim work.fifo_tb
run -all
```



```
ModelSim> vlog -sv rtl/sync_fifo.sv
# Model Technology ModelSim - Intel FPGA Edition vlog 2020.1 Compiler 2020.02 Feb 28 2020
# Start time: 10:31:40 on Jul 26,2026
# vlog -reportprogress 300 -sv rtl/sync_fifo.sv
# -- Compiling module sync_fifo
#
# Top level modules:
#   sync_fifo
# End time: 10:31:40 on Jul 26,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
ModelSim> vlog -sv tb/fifo_if.sv
# Model Technology ModelSim - Intel FPGA Edition vlog 2020.1 Compiler 2020.02 Feb 28 2020
# Start time: 10:31:40 on Jul 26,2026
# vlog -reportprogress 300 -sv tb/fifo_if.sv
# -- Compiling interface fifo_if
#
# Top level modules:
#   --none--
# End time: 10:31:40 on Jul 26,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0

ModelSim> vlog -sv -mfcu \
    tb/fifo_transaction.sv \
    tb/fifo_monitor.sv \
    tb/fifo_scoreboard.sv \
    tb/fifo_tb.sv
# Model Technology ModelSim - Intel FPGA Edition vlog 2020.1 Compiler 2020.02 Feb 28 2020
# Start time: 10:31:40 on Jul 26,2026
# vlog -reportprogress 300 -sv -mfcu tb/fifo_transaction.sv tb/fifo_monitor.sv tb/fifo_scoreboard.sv tb/fifo_tb.sv
# -- Compiling package fifo_transaction_sv_unit
# -- Compiling module fifo_tb
#
# Top level modules:
#   fifo_tb
# End time: 10:31:40 on Jul 26,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
```

Hình 7: Đoạn lệnh compile đúng thứ tự với tùy chọn `-mfcu`.

## Kết quả sau khi sửa

ModelSim nhận diện đầy đủ class, interface và module. Testbench elaboration thành công và mô phỏng bình thường.



### Bài học rút ra

- Thứ tự compile và phạm vi nhìn thấy (scope visibility) của class là yếu tố then chốt trong SystemVerilog.
- Việc một file đã compile trước không đồng nghĩa class trong đó tự động nhìn thấy từ compilation unit khác.

# LOG ENTRY #03: Ineffective Scoreboard Delay Fix

## ISSUE METADATA & TRACKING INFO

**Date Logged:** 24/07/2026  
**Issue ID:** BUG-20260724-03  
**Component:** Scoreboard Architecture / Timing  
**Severity:** Medium (Conceptual & Architectural Error)  
**Status:** Resolved

## Bối cảnh

Sau khi phát hiện dữ liệu đọc bị lấy mẫu quá sớm ở Log Entry #01, một giải pháp ban đầu được thử nghiệm là giữ nguyên Monitor và thêm delay bên trong Scoreboard trước khi so sánh `actual` và `expected`.

## Hiện tượng

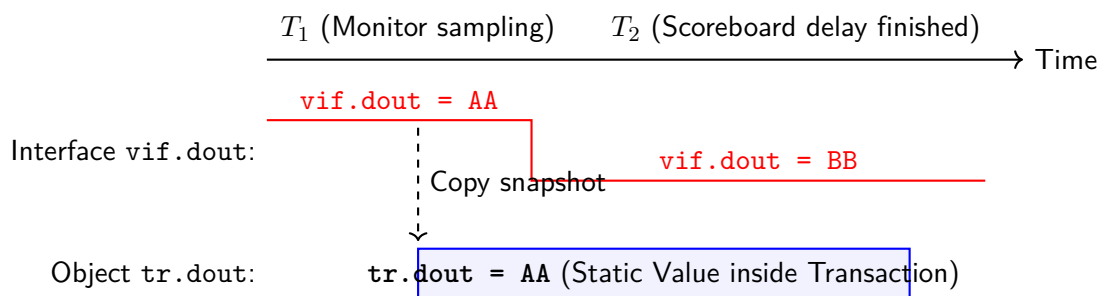
Dù Scoreboard chờ thêm một khoảng thời gian (`#5ns` hoặc `@(posedge clk)`), giá trị `actual` vẫn bị sai và không hề thay đổi. Scoreboard vẫn so sánh dữ liệu cũ.

## Nguyên nhân

Transaction là một **snapshot (ảnh chụp cố định)** của tín hiệu tại thời điểm Monitor thực hiện gán:

```
tr.dout = vif.dout;  
mon2scb.put(tr);
```

Sau khi `tr.dout` được gán, dữ liệu nằm trong biến này hoàn toàn độc lập và không hề liên kết với tín hiệu `vif.dout` trên interface. Việc Scoreboard hoãn xử lý chỉ hoãn thời điểm so sánh chứ không thay đổi được nội dung bức ảnh snapshot đã chụp sai từ trước.



Hình 8: Minh họa sự khác biệt giữa tín hiệu động trên Interface và dữ liệu tĩnh trong Transaction.

## Cách khắc phục

Xóa bỏ toàn bộ delay trong Scoreboard. Thời điểm lấy mẫu phải được sửa triệt để ngay tại Monitor.

1. Monitor chờ dout ổn định mới gán `tr.dout = vif.dout` rồi gửi qua mailbox.
2. Scoreboard tuân thủ đúng chức năng: Nhận transaction → Cập nhật Reference Model → So sánh và Report PASS/FAIL.

## Kết quả sau khi sửa

Transaction chứa đúng dữ liệu ngay khi tạo ra. Scoreboard chạy mượt mà không cần thêm bất kỳ lệnh delay nào.

### Bài học rút ra

Phân định rõ trách nhiệm (Separation of Concerns) trong Layered Testbench:

- **Monitor:** Chịu trách nhiệm hoàn toàn về Timing và lấy mẫu tín hiệu đúng thời điểm.
- **Scoreboard:** Chịu trách nhiệm kiểm tra logic dữ liệu.
- Một khi Transaction đã thu thập sai dữ liệu, thì hoãn xử lý ở Scoreboard không thể khôi phục dữ liệu đúng.

## LOG ENTRY #04: Ignored Rejected Operations in Accepted-Only Scoreboard

### ISSUE METADATA & TRACKING INFO

|                     |                                         |
|---------------------|-----------------------------------------|
| <b>Date Logged:</b> | 25/07/2026                              |
| <b>Issue ID:</b>    | BUG-20260725-04                         |
| <b>Component:</b>   | Monitor / Scoreboard / Protocol Checker |
| <b>Severity:</b>    | Medium (Test Coverage Hole)             |
| <b>Status:</b>      | Resolved                                |

### Bối cảnh

Monitor ban đầu chỉ tạo transaction khi một thao tác thực sự được DUT chấp nhận:

```
wr_fire = wr_en && !full;  
rd_fire = rd_en && !empty;
```

Monitor chỉ gửi transaction đến scoreboard khi `wr_fire` hoặc `rd_fire` bằng 1.

### Hiện tượng

Trong test case **Read when empty**, testbench kích hoạt `rd_en = 1` lúc FIFO đang **empty**. Tương tự với **Write when full**, kích hoạt `wr_en = 1` lúc `full = 1`. Tối ưu là DUT phải chặn thao tác. Tuy nhiên, Scoreboard hoàn toàn không in bất kỳ thông báo PASS hay FAIL nào cho các sự kiện này.

```

#
# =====
#                               SYNC FIFO VERIFICATION
#                               =====
# DATA_WIDTH = 8
# DEPTH = 16
# -----
#
# -----
#                               TEST CASE 2: EMPTY READ
# -----
#
# =====
#                               TEST SUMMARY
# =====
# Writes      : 0
# Reads       : 0
# Passed      : 0
# Mismatches  : 0
# Queue.size  : 0
# Ref Queue   : '{}'
# -----
# RESULT      : TEST PASSED
# =====
# ** Note: $finish      : tb/fifo_tb.sv(254)
#      Time: 80 ns  Iteration: 1  Instance: /fifo_tb

```

Hình 9: Console log thể hiện testbench có phát request nhưng Monitor không gửi transaction.

## Nguyên nhân

Khi FIFO empty, `rd_fire = 1 && 0 = 0`. Do `rd_fire = 0`, Monitor bỏ qua và không tạo transaction gửi qua Mailbox. Scoreboard thụ động chỉ xử lý những gì nhận từ Mailbox nên hoàn toàn "mù" trước ý định kích hoạt yêu cầu của testbench.

## Ý nghĩa & Phân tích

Scoreboard thông thường được thiết kế để kiểm tra dữ liệu của **Accepted Transactions**. Các thao tác bị từ chối (**Rejected Operations**) như `*Read when empty*` hay `*Write when full*` thuộc nhóm kiểm tra giao thức (protocol/corner-case checking). Nếu chỉ quan sát `fire`, testbench sẽ bỏ sót coverage của các góc cấm này.

## Cách kiểm tra và xử lý hiện tại

Ở giai đoạn này, testbench chưa sử dụng SystemVerilog Assertions để tự động kiểm tra thao tác đọc khi FIFO rỗng. Thay vào đó, corner case này được kiểm tra thủ công thông qua stimulus và waveform.

Test case được xây dựng theo trình tự:

1. Reset FIFO để đưa hệ thống về trạng thái ban đầu.
2. Ghi một giá trị 8'hAA vào FIFO.
3. Thực hiện lệnh đọc lần thứ nhất để lấy phần tử 8'hAA ra khỏi FIFO.
4. Thực hiện lệnh đọc lần thứ hai khi FIFO đã rỗng.
5. Quan sát waveform để xác nhận rằng chỉ lần đọc thứ nhất được chấp nhận, còn lần đọc thứ hai bị FIFO từ chối.

```
task automatic tc2_empty_read();
    reset_dut();

    // Ghi mot phan tu vao FIFO
    write_data(8'hAA);

    // Lan doc thu nhat: hop le, doc du lieu AA
    read_data();

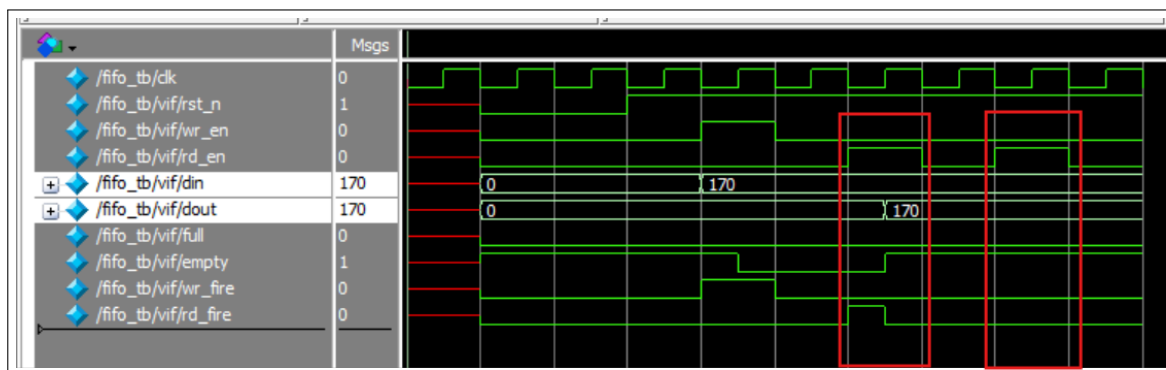
    // Lan doc thu hai: FIFO da empty, yeu cau bi tu choi
    read_data();
endtask
```

Trong waveform, lần đọc thứ nhất xảy ra khi `empty = 0`, do đó điều kiện giao dịch đọc hợp lệ:

$$rd\_fire = rd\_en \&\& !empty$$

nhận giá trị bằng 1 và dữ liệu 8'hAA được đọc ra khỏi FIFO.

Sau lần đọc này, FIFO chuyển sang trạng thái rỗng và tín hiệu `empty` được đưa lên 1. Khi lệnh `read_data()` thứ hai tiếp tục kích hoạt `rd_en`, điều kiện `rd_fire` vẫn bằng 0 vì `empty = 1`. Điều đó chứng tỏ DUT không chấp nhận thao tác đọc thứ hai.



Hình 10: Waveform của test case Empty Read: lần đọc thứ nhất được chấp nhận, trong khi lần đọc thứ hai bị từ chối do FIFO đã rỗng.

## Kết quả kiểm tra

Waveform cho thấy chỉ có một giao dịch đọc hợp lệ được thực hiện sau khi ghi giá trị 8'hAA. Ở yêu cầu đọc tiếp theo, `rd_en` được kích hoạt nhưng `rd_fire` không được tạo ra vì tín hiệu `empty` đang ở mức 1.

Kết quả này xác nhận FIFO đã từ chối thao tác đọc khi rỗng đúng với thiết kế. Tuy nhiên, việc kiểm tra hiện tại vẫn phụ thuộc vào quan sát waveform và chưa được tự động hóa bằng Assertion.

### Bài học kinh nghiệm

Một test case đọc khi FIFO rỗng cần tạo được rõ ràng cả hai trường hợp: một lần đọc hợp lệ và một lần đọc bị từ chối.

Việc ghi một phần tử rồi thực hiện hai lần đọc liên tiếp giúp waveform thể hiện trực quan sự khác biệt giữa một giao dịch được chấp nhận và một yêu cầu bị chặn bởi trạng thái `empty`.

# LOG ENTRY #05: Simultaneous Read/Write Timing Alignment Issue

## ISSUE METADATA & TRACKING INFO

|              |                                                     |
|--------------|-----------------------------------------------------|
| Date Logged: | 26/07/2026                                          |
| Issue ID:    | BUG-20260726-05                                     |
| Component:   | Stimulus Generator / Sequence Task                  |
| Severity:    | Minor (False Positive Test / Insufficient Stimulus) |
| Status:      | Resolved                                            |

## Bối cảnh

Test case **Simultaneous Read/Write** được tạo ra để kiểm tra khả năng FIFO thực hiện đồng thời một thao tác ghi và một thao tác đọc trong cùng một chu kỳ clock (**wr\_en** = 1 và **rd\_en** = 1 tại cùng posedge clk).

```
task automatic tc5_simultaneous_rw();
    $display("-----");
    $display(" TEST CASE 5: Simultaneous READ / WRITE");
    $display("-----");
    reset_dut(); @(negedge vif.clk);
    write_data(8'hAA);
    write_data(8'hBB);
    write_data(8'hCC);

    @(negedge vif.clk);
    write_data(8'hDD);
    read_data(); read_data();

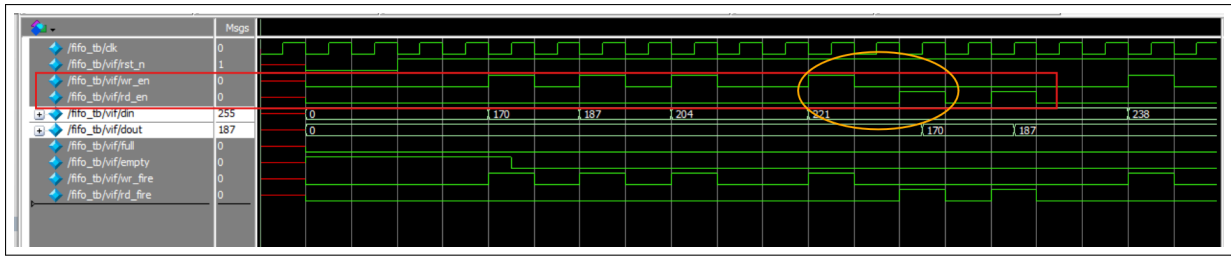
    @(negedge vif.clk);
    write_data(8'hEE);
    write_data(8'hFF);

endtask
```

## Hiện tượng

Test case báo PASS, nhưng khi soi Waveform thì thấy **wr\_en** và **rd\_en** không hề cùng bằng 1 tại bất kỳ cạnh lên clock nào. Tín hiệu này bật ở chu kỳ trước, bị hạ xuống rồi tín hiệu kia mới bật ở chu kỳ sau.





Hình 11: Waveform lỗi cho thấy wr\_en và rd\_en bị lệch nhau một chu kỳ.

## Nguyên nhân

Các task `write_data()` và `read_data()` được thiết kế tuần tự, mỗi task tự quản lý enable và tự chờ cạnh clock:

```
write_data(data); // Chờ 1 clock cycle để hoàn thành
read_data();      // Chạy ở clock cycle tiếp theo
```

Khi gọi nối tiếp, thao tác ghi hoàn thành xong mới đến thao tác đọc. Dù dùng `fork...join`, nếu cơ chế chờ clock bên trong 2 task lệch nhau thì tín hiệu vẫn không trùng khớp tại 1 xung nhịp.

## Cách khắc phục

Điều khiển trực tiếp `wr_en` và `rd_en` trong cùng một chu kỳ clock, thay vì gọi các task tuần tự. Cụ thể, trong test case, sau khi reset và ghi dữ liệu ban đầu, trực tiếp drive cả hai enable cùng lúc:

```

task automatic tc5_simultaneous_rw();

    reset_dut();

    @(negedge vif.clk);
    write_data(8'hAA);
    write_data(8'hBB);
    write_data(8'hCC);

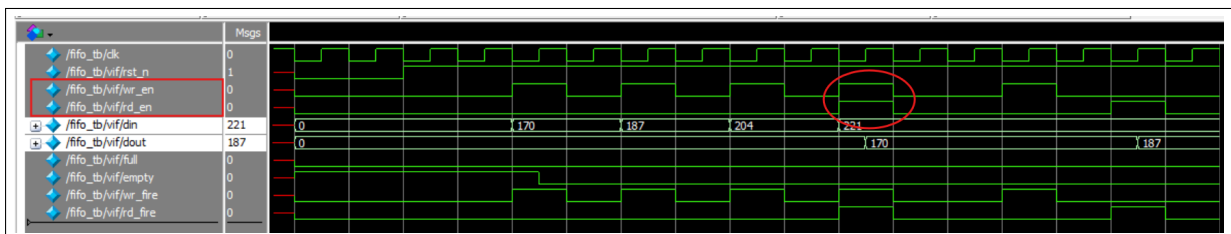
    // Drive trực tiếp wr_en và rd_en thì mỗi có xung cùng lúc
    @(negedge vif.clk);
    vif.din = 8'hDD;
    vif.wr_en = 1;
    vif.rd_en = 1;

    @(negedge vif.clk);
    vif.wr_en = 0;
    vif.rd_en = 0;

    @(negedge vif.clk);
    write_data(8'hDD);
    read_data();

endtask

```



Hình 12: Waveform sau khi sửa: wr\_en và rd\_en cùng bằng 1 tại cùng một posedge clk.

## Kết quả sau khi sửa

Monitor ghi nhận cả Write transaction và Read transaction trong cùng 1 chu kỳ. Reference queue vừa pop phần tử cũ vừa push phần tử mới chính xác.

### Bài học rút ra

Một Test Case báo PASS không có nghĩa là corner case mong muốn đã thực sự được kích hoạt!

- Tên task hay tên test case không đảm bảo tín hiệu tạo ra đúng thời điểm.
- Luôn luôn đối chiếu sóng Waveform hoặc dùng Assertion để kiểm chứng stimulus đã tạo đúng điều kiện biên (Corner Case) hay chưa.